Formal Methods in Software Engineering

First-Order Logic, Spring 2026

Konstantin Chukharev

Introduction to FOL

Motivation: The Limits of Propositional Logic

Propositional logic was the first widely used formal calculus of reasoning (Boole, 1847; Frege, 1879).

But it has fundamental *limitations*:

What PL cannot express:

- "All humans are mortal" (quantification)
- "Every even number > 2 is a sum of two primes" (Goldbach)
- "For all $\varepsilon > 0$, there exists $\delta > 0$..." (ε–δ proofs)
- "The array has no out-of-bounds access" (program verification)

Why this matters:

- Mathematics *needs* universal statements
- Program specifications quantify over inputs
- Database queries select objects satisfying properties
- Scientific theories make generalizations

What is First-Order Logic? The Historical Context

First-order logic emerged from three converging streams in late 19th/early 20th century:

Frege (1879): *Begriffsschrift* — the first formal system with quantifiers. Goal: foundations for arithmetic. Notation was 2D and hard to use.

Russell (1903): *Principia Mathematica* (with Whitehead, 1910-1913). Derived mathematics from logic. Discovered Russell's paradox: naive set theory is inconsistent.

Hilbert (1920s): Hilbert's program: prove consistency of mathematics via finitary methods. Pushed for formal axiomatization. Goal shattered by Gödel (1931).

Why "first-order"? The quantifiers $\forall x, \exists x$ range over *individuals* (elements of the domain).
In *second-order* logic, we could quantify over *sets* or *predicates*: $\forall P. \exists x.\, P(x)$.
In *higher-order* logic (used in Coq, Lean, Isabelle), we quantify over functions of functions, *etc.*

First-order logic is expressive and has strong metatheory (soundness, completeness, compactness).

Syntax: The Language of FOL

Similar to PL, first-order logic is a formal system with a *syntax* and *semantics*.

First-order logic is an umbrella term for different *first-order languages*.
Each language is specified by its *signature* Σ — the vocabulary of function and relation symbols.

Symbols of a first-order language are divided into *logical symbols* (fixed) and *non-logical parameters* (signature-dependent):

Logical symbols (universal):

- Parentheses: $($, $)$
- Logical connectives: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\iff$
- Variables: x, y, z, ... (infinite supply)
- Quantifiers: $\forall$ ("for all") and $\exists$ ("there exists")
- Equality: $=$

Non-logical parameters (signature Σ):

- Constants: *e.g.*, 0, $\emptyset$, e (arity 0 functions)
- Functions: *e.g.*, S (successor), $+$, $\times$, $f(x, y)$
- Predicates: *e.g.*, $<$, $\in$, $\mathrm{Prime}(x)$, $P(x, y)$

Each symbol has a fixed *arity* (number of arguments).

Note: The pair $\wedge$, $\neg$ is functionally complete for Boolean connectives. Also, $\forall$ alone is sufficient for quantification since $\exists x.\varphi \equiv \neg\forall x.\neg\varphi$.

Predicates and Functions

Predicates are used to express properties or relations among objects.

Functions are similar to predicates but return a value, not necessarily a truth value.

Each predicate and function symbol has a fixed *arity* (number of arguments).

- Equality is a special predicate with arity 2.
- Constants can be seen as functions with arity 0.

First-Order Languages

A first-order language is specified by its *parameters*.

Propositional logic:
- Equality: *no*
- Constants: *none*
- Predicates: A_1, A_2, ...
- Functions: *none*

Set theory:
- Equality: *yes*
- Constants: $\emptyset$
- Predicates: $\in$
- Functions: *none*

Number theory:
- Equality: *yes*
- Constants: 0
- Predicates: $<$
- Functions: S (successor), $+$, $\times$

FOL Syntax

Signatures

Definition 1 (Signature): A *first-order signature* $\Sigma = \langle \mathcal{F}, \mathcal{R} \rangle$ consists of:

- A set of *function symbols* $\mathcal{F}$, each with an arity $n \geq 0$.
- A set of *relation symbols* (predicates) $\mathcal{R}$, each with an arity $n \geq 1$.

Functions of arity 0 are *constants*.

We assume a countably infinite set of *variables* $\mathcal{V} = \{x, y, z, x_1, x_2, \ldots\}$.

Example: **Arithmetic:** $\Sigma = \langle \{0, S, +, \times\}, \{<, =\} \rangle$
where 0 is a constant, S is unary, $+, \times$ are binary functions; $<, =$ are binary relations.

Example: **Graph theory:** $\Sigma = \langle \emptyset, \{\mathrm{Edge}, \mathrm{Path}\} \rangle$
where Edge and Path are binary predicates (no function symbols).

Terms and Formulas

Definition 2 (Term): *Terms* over signature Σ are defined inductively:

- Every variable $x \in \mathcal{V}$ is a term.
- If $f \in \mathcal{F}$ has arity n and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.

Definition 3 (Formula): *Formulas* over Σ are defined inductively:

- If $R \in \mathcal{R}$ has arity n and $t_1, \ldots, t_n$ are terms, then $R(t_1, \ldots, t_n)$ is an *atomic formula* (atom).
- If φ, ψ are formulas: $\neg\varphi$, $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \rightarrow \psi)$, $(\varphi \Longleftrightarrow \psi)$ are formulas.
- If φ is a formula and $x \in \mathcal{V}$: $\forall x.\, \varphi$ and $\exists x.\, \varphi$ are formulas.

Example: $\forall x.\, (x = 0 \vee \exists y.\, S(y) = x)$ — "every natural number is 0 or a successor."

Free and Bound Variables

The *scope* of $\forall x$ (or $\exists x$) is the immediately following subformula.

Definition 4 (Free Variables): The set $\mathrm{FV}(\varphi)$ of *free variables* of φ is defined inductively:

- $\mathrm{FV}(R(t_1, ..., t_n)) = \mathrm{Vars}(t_1) \cup ... \cup \mathrm{Vars}(t_n)$
- $\mathrm{FV}(\neg\varphi) = \mathrm{FV}(\varphi)$; $\quad \mathrm{FV}(\varphi \circ \psi) = \mathrm{FV}(\varphi) \cup \mathrm{FV}(\psi)$ for $\circ \in \{\wedge, \vee, \rightarrow, \Longleftrightarrow\}$
- $\mathrm{FV}(\forall x.\, \varphi) = \mathrm{FV}(\exists x.\, \varphi) = \mathrm{FV}(\varphi) \setminus \{x\}$

A formula with no free variables is a *sentence* (closed formula).

Example: In $\forall x.\, P(x, y)$: x is *bound* (in scope of $\forall x$), y is *free*. Not a sentence.

Note: Only *sentences* have a definite truth value in a structure. Formulas with free variables are like "open predicates" — they become true/false once you fix values for the free variables.

Substitution

Definition 5 (Substitution): $\varphi[t/x]$ denotes the formula obtained by replacing every *free* occurrence of x with term t.

A substitution $[t/x]$ is *capture-free* in φ if no free variable in t becomes bound.

Example: $\forall y.\,(x < y)\,[S(0)/x] = \forall y.\,(S(0) < y)$ — correct substitution.

$\exists y.\,(x < y)\,[y + 1/x] = \exists y.\,(y + 1 < y)$ — **variable capture!**
The free y in the substituted term becomes bound. Fix: rename bound variable first: $\exists z.\,(y + 1 < z)$.

Variable capture: Always check for name clashes before substituting. Rename bound variables if necessary.

FOL Semantics

Structures (Models)

Definition 6 (Structure (Model)): A *structure* $\mathfrak{A}$ for signature $\Sigma = \langle \mathcal{F}, \mathcal{R} \rangle$ consists of:

- A non-empty *domain* (universe) A — the set of objects we reason about.
- For each n-ary function symbol $f \in \mathcal{F}$: a function $f^{\mathfrak{A}} : A^n \to A$.
- For each n-ary relation symbol $R \in \mathcal{R}$: a relation $R^{\mathfrak{A}} \subseteq A^n$.

A *variable assignment* $\sigma : \mathcal{V} \to A$ maps each variable to a domain element.

Example: For the arithmetic signature $\Sigma = \langle \{0, S, +, \times\}, \{<, =\} \rangle$:

- $\mathfrak{N} = (\mathbb{N}, 0, S, +, \times, <, =)$ — the *standard* (intended) model.
- $\mathfrak{Z} = (\mathbb{Z}, 0, S, +, \times, <, =)$ — a different, equally valid structure.
- The sentence $\exists m. \forall x. m \leq x$ is true in $\mathfrak{N}$ (least element exists: $m = 0$) but *false* in $\mathfrak{Z}$ (integers have no least element).

The same sentence may be true in one structure and false in another.
Validity means truth in *all* structures.

Evaluating FOL Formulas

Given a structure $\mathfrak{A}$ and a variable assignment σ, truth is defined inductively. The main semantic difference from PL is that quantifiers range over domain elements.

Definition 7 (Satisfaction Relation): The relation $\mathfrak{A}, \sigma \vDash \varphi$ is defined inductively:

- $\mathfrak{A}, \sigma \vDash R(t_1, \ldots, t_n)$ iff $\left(t_1^{\mathfrak{A},\sigma}, \ldots, t_n^{\mathfrak{A},\sigma}\right) \in R^{\mathfrak{A}}$
- Boolean connectives: as in PL.
- $\mathfrak{A}, \sigma \vDash \forall x.\, \varphi$ iff $\mathfrak{A}, \sigma[x \mapsto a] \vDash \varphi$ for *every* $a \in A$.
- $\mathfrak{A}, \sigma \vDash \exists x.\, \varphi$ iff $\mathfrak{A}, \sigma[x \mapsto a] \vDash \varphi$ for *some* $a \in A$.

Here $\sigma[x \mapsto a]$ maps x to a and agrees with σ on all other variables.

For *sentences* (no free variables), the truth value depends only on the structure: we write $\mathfrak{A} \vDash \varphi$.

Example: Let $\mathfrak{A} = \left(\{1, 2, 3\}, <^{\mathfrak{A}} = \{(1, 2), (1, 3), (2, 3)\}\right)$ be a structure for signature $\{<\}$.

Evaluate $\forall x.\, \exists y.\, x < y$ in $\mathfrak{A}$:

- $x = 1$: need y with $1 < y$. Take $y = 2$. ✓

Evaluating FOL Formulas [2]

- $x = 2$: need y with $2 < y$. Take $y = 3$. ✓
- $x = 3$: need y with $3 < y$. No such y in $\{1, 2, 3\}$. ✗

Result: $\mathfrak{A} \not\models \forall x.\, \exists y.\, x < y$. The formula is *falsified* by the element 3.

But $\forall x.\, \exists y.\, x < y$ *is* satisfied in $\mathfrak{N} = (\mathbb{N}, <)$ — for every n, take $y = n + 1$.

Validity and Satisfiability in FOL

Definition 8: Let φ be an FOL sentence.

- φ is *valid* ($\models \varphi$) if $\mathfrak{A} \models \varphi$ for *every* structure $\mathfrak{A}$.
- φ is *satisfiable* if $\mathfrak{A} \models \varphi$ for *some* structure $\mathfrak{A}$.
- φ is *unsatisfiable* if no structure satisfies it.

Difference from PL:

- In PL, the space of interpretations is *finite* (2^n truth assignments).
- In FOL, structures may be infinite and of arbitrary cardinality — this is the source of undecidability in general validity/satisfiability problems.

Example:

- $\forall x.\, (P(x) \lor \neg P(x))$ — valid (instance of LEM, holds in every structure).
- $\forall x.\, P(x) \land \exists x.\, \neg P(x)$ — unsatisfiable.
- $\exists x.\, P(x)$ — satisfiable (in any non-empty structure with P non-empty) but not valid.

Quantifier Equivalences

Many useful equivalences govern quantifiers:

De Morgan duality for quantifiers:

- $\neg\forall x.\, \varphi \equiv \exists x.\, \neg\varphi$
- $\neg\exists x.\, \varphi \equiv \forall x.\, \neg\varphi$

Distribution:

- $\forall x.\, (\varphi \land \psi) \equiv (\forall x.\, \varphi) \land (\forall x.\, \psi)$
- $\exists x.\, (\varphi \lor \psi) \equiv (\exists x.\, \varphi) \lor (\exists x.\, \psi)$

Vacuous quantification (if x not free in ψ):

- $\forall x.\, \psi \equiv \psi, \quad \exists x.\, \psi \equiv \psi$

Commutativity of like quantifiers:

- $\forall x.\, \forall y.\, \varphi \equiv \forall y.\, \forall x.\, \varphi$
- $\exists x.\, \exists y.\, \varphi \equiv \exists y.\, \exists x.\, \varphi$

Prenex pulling (if x not free in ψ):

- $\psi \land \forall x.\, \varphi \equiv \forall x.\, (\psi \land \varphi)$
- $\psi \lor \exists x.\, \varphi \equiv \exists x.\, (\psi \lor \varphi)$

Quantifier Equivalences [2]

In general, $\exists x.\, \forall y.\, R(x, y)$ is *stronger* than $\forall y.\, \exists x.\, R(x, y)$: the first requires one uniform witness for all y.

Example: Let $R(x, y) \equiv$ "x is the parent of y". Then:

- $\exists x.\, \forall y.\, R(x, y)$ = "someone is the parent of *everyone*" — false.
- $\forall y.\, \exists x.\, R(x, y)$ = "everyone *has* a parent" — true.

In general: $\exists\forall \vDash \forall\exists$, but $\forall\exists \nvDash \exists\forall$.

Prenex Normal Form

Definition 9 (Prenex Normal Form (PNF)): A formula is in *prenex normal form* if it has the shape:

$$Q_1 x_1 . Q_2 x_2 . \ldots Q_n x_n . \psi$$

where each $Q_i \in \{\forall, \exists\}$ and ψ is *quantifier-free* (the *matrix*).

Theorem 1: Every FOL formula can be converted to an equivalent formula in PNF (after renaming bound variables to avoid capture if necessary).

Example: $(\forall x . P(x)) \to (\exists y . Q(y))$
$\equiv \neg \forall x . P(x) \vee \exists y . Q(y)$ *(eliminate $\to$)*
$\equiv \exists x . \neg P(x) \vee \exists y . Q(y)$ *(quantifier duality)*

PNF separates the *quantifier prefix* (alternation structure) from the *propositional skeleton* (matrix).
The alternation depth ($\forall\exists\forall\ldots$) determines complexity.

FOL Proofs

FOL Proof Rules: Quantifiers

$\forall$-introduction ($\forall$i):
Derive $\varphi(y)$ for an *arbitrary* y.
Conclude $\forall x.\, \varphi(x)$.

1	$\varphi(y)$	$\vdots$ *(y arbitrary)*
2	$\forall x.\, \varphi(x)$	$\forall i$ *1*

$\exists$-introduction ($\exists$i):
From $\varphi(t)$ for some term t, conclude $\exists x.\, \varphi(x)$.

1	$\varphi(t)$	*Premise*
2	$\exists x.\, \varphi(x)$	$\exists i$ *1*

$\forall$-elimination ($\forall$e):
From $\forall x.\, \varphi(x)$, conclude $\varphi(t)$ for *any* term t.

1	$\forall x.\, \varphi(x)$	*Premise*
2	$\varphi(t)$	$\forall e$ *1*

$\exists$-elimination ($\exists$e):
From $\exists x.\, \varphi(x)$, open a subproof assuming $\varphi(y)$ for *fresh* y, derive ψ. Conclude ψ.

1	$\exists x.\, \varphi(x)$	*Premise*
2	$\quad \varphi(y)$	*y fresh*
3	$\quad \vdots$	
4	$\quad \psi$	$\vdots$
5	ψ	$\exists e$ *1, 2–4*

FOL Proof Rules: Quantifiers [2]

Side conditions:

- $\forall$i: y must be *arbitrary* — not free in any undischarged assumption.
- $\forall$e: t can be any term (universal *instantiation*).
- $\exists$i: give a *witness* term t.
- $\exists$e: y must be *fresh* — not free in ψ or any undischarged assumption besides $\varphi(y)$.

FOL Proof: A Complete Fitch Example

Example: **Prove:** $\forall x.\,(P(x) \to Q(x)), \exists x.\,P(x) \vdash \exists x.\,Q(x)$

"If every P is a Q, and some P exists, then some Q exists."

1	$\forall x.\,(P(x) \to Q(x))$	*Premise*
2	$\exists x.\,P(x)$	*Premise*
3	$\quad P(a)$	*a fresh*
4	$\quad P(a) \to Q(a)$	*$\forall$e 1*
5	$\quad Q(a)$	*$\to$e 3, 4*
6	$\quad \exists x.\,Q(x)$	*$\exists$i 5*
7	$\exists x.\,Q(x)$	*$\exists$e 2, 3–6*

Line 3: open $\exists$e subproof — name the witness a (fresh).
Line 4: instantiate $\forall x.\,(P(x) \to Q(x))$ with a ($\forall$e).
Line 5: modus ponens on lines 3 and 4.
Line 6: from $Q(a)$, existentially generalize ($\exists$i).
Line 7: close $\exists$e — the conclusion $\exists x.\,Q(x)$ does not mention a, so it survives.

Semantic Arguments for FOL

The rules below are the propositional tableau core.
Full FOL tableaux add quantifier rules (instantiation and witness introduction):

$$\text{(a)}\ \frac{\mathcal{J} \vDash \neg\alpha}{\mathcal{J} \nvDash \alpha}$$

$$\text{(b)}\ \frac{\mathcal{J} \nvDash \neg\alpha}{\mathcal{J} \vDash \alpha}$$

$$\text{(c)}\ \frac{\mathcal{J} \vDash \alpha \wedge \beta}{\mathcal{J} \vDash \alpha, \mathcal{J} \vDash \beta}$$

$$\text{(d)}\ \frac{\mathcal{J} \nvDash \alpha \wedge \beta}{\mathcal{J} \nvDash \alpha \mid \mathcal{J} \nvDash \beta}$$

$$\text{(e)}\ \frac{\mathcal{J} \vDash \alpha \vee \beta}{\mathcal{J} \vDash \alpha \mid \mathcal{J} \vDash \beta}$$

$$\text{(f)}\ \frac{\mathcal{J} \nvDash \alpha \vee \beta}{\mathcal{J} \nvDash \alpha, \mathcal{J} \nvDash \beta}$$

$$\text{(i)}\ \frac{\mathcal{J} \vDash \alpha \qquad \mathcal{J} \nvDash \alpha}{\mathcal{J} \vDash \bot}$$

$$\text{(g)}\ \frac{\mathcal{J} \vDash \alpha \to \beta}{\mathcal{J} \nvDash \alpha \mid \mathcal{J} \vDash \beta}$$

$$\text{(h)}\ \frac{\mathcal{J} \nvDash \alpha \to \beta}{\mathcal{J} \vDash \alpha, \mathcal{J} \nvDash \beta}$$

$$\text{(j)}\ \frac{\mathcal{J} \vDash \alpha \Longleftrightarrow \beta}{\mathcal{J} \vDash \alpha, \mathcal{J} \vDash \beta \mid \mathcal{J} \nvDash \alpha, \mathcal{J} \nvDash \beta}$$

$$\text{(k)}\ \frac{\mathcal{J} \nvDash \alpha \Longleftrightarrow \beta}{\mathcal{J} \nvDash \alpha, \mathcal{J} \vDash \beta \mid \mathcal{J} \vDash \alpha, \mathcal{J} \nvDash \beta}$$

Metatheorems and the Limits of Logic

Metatheory Roadmap: From Syntax to Models

Up to now, we used FOL *inside* proofs and specifications. Now we step back and analyze FOL *itself*.

Syntactic side:

- Derivability: $T \vdash \varphi$
- Consistency: $T \nvdash \bot$
- Proof rules and finite derivations

Semantic side:

- Satisfaction: $\mathfrak{M} \vDash \varphi$
- Satisfiability: some model exists
- Consequence: $T \vDash \varphi$

Lindenbaum extension $\Rightarrow$ Henkin witnesses $\Rightarrow$ term model $\Rightarrow$ Truth Lemma $\Rightarrow$ Completeness.

Then we get Compactness and Löwenheim–Skolem almost for free.

Consequence, Consistency, Satisfiability

Definition 10: Let T be a set of FOL sentences and φ a sentence.

- $T \vDash \varphi$ means: every model of T is a model of φ.
- $T \vdash \varphi$ means: φ is derivable from T.
- T is *consistent* iff $T \nvdash \bot$.
- T is *satisfiable* iff some structure $\mathfrak{M}$ satisfies every sentence in T.

Soundness gives $T \vdash \varphi \to T \vDash \varphi$.

Completeness will prove the converse: $T \vDash \varphi \to T \vdash \varphi$.

Step 1: Lindenbaum Extension

Theorem 2 (Lindenbaum Lemma): If T is consistent, then there exists a *maximally consistent* theory T^* with $T \subseteq T^*$. Maximal means: for every sentence ψ, either $\psi \in T^*$ or $\neg\psi \in T^*$.

Proof *(idea)*: Enumerate sentences as $\psi_0, \psi_1, \psi_2, \ldots$ and build $T_0 \subseteq T_1 \subseteq \ldots$ with $T_0 = T$.

At stage n:

- if $T_n \cup \{\psi_n\}$ is consistent, set $T_{n+1} = T_n \cup \{\psi_n\}$;
- otherwise set $T_{n+1} = T_n \cup \{\neg\psi_n\}$.

Let $T^* = T_0 \cup T_1 \cup \ldots$.

Any proof uses finitely many premises, so any contradiction would already appear at some finite stage. Hence consistency is preserved, and every sentence is decided. $\square$

Step 2: Henkinization (Adding Witnesses)

Maximal consistency alone does *not* guarantee explicit witnesses for existential sentences.

Definition 11 (Henkin Property): A theory T^* is *Henkin* if whenever $\exists x.\, \psi(x) \in T^*$, there is a constant c_ψ such that $\psi(c_\psi) \in T^*$.

Construction idea: For each existential sentence $\exists x.\, \psi(x)$, introduce a fresh constant c_ψ and add a witness condition.

Consistency argument: Add witness only when the existential branch is consistent. Freshness of c_ψ ensures the new name does not interact with earlier formulas.

If $T_n \cup \{\exists x.\, \psi(x)\}$ is consistent, then $T_n \cup \{\exists x.\, \psi(x), \psi(c)\}$ is consistent for fresh c.

Step 3: Canonical Term Model

Assume T^* is maximally consistent and Henkin.

Definition 12 (Term Model): Work in the extended language L_H with Henkin constants.

Domain: equivalence classes $[t]$ of closed terms under $t \sim s \iff T^* \vdash t = s$.

Interpret symbols by syntax:

- constants: $c^{\mathfrak{M}} = [c]$;
- functions: $f^{\mathfrak{M}}([t_1], ..., [t_n]) = [f(t_1, ..., t_n)]$;
- relations: $\mathfrak{M} \vDash R([t_1], ..., [t_n])$ iff $R(t_1, ..., t_n) \in T^*$.

Note: Equality axioms/substitutivity ensure these interpretations are well-defined.

Step 4: Truth Lemma

Theorem 3 (Truth Lemma): For every formula $\varphi(x_1, ..., x_k)$ and closed terms $t_1, ..., t_k$:

$$\mathfrak{M} \vDash \varphi([t_1], ..., [t_k]) \quad \Longleftrightarrow \quad \varphi(t_1, ..., t_k) \in T^*$$

.

In particular, for every sentence φ: $\mathfrak{M} \vDash \varphi \Longleftrightarrow \varphi \in T^*$.

Proof *(sketch)*: By structural induction on φ:

- atomic case by definition of the term model;
- Boolean connectives by maximal consistency;
- existential case uses the Henkin witness;
- universal case follows by duality: $\forall x.\, \psi \equiv \neg \exists x.\, \neg \psi$.

$\square$

Completeness Theorem (Gödel, 1930)

Theorem 4 (Completeness): For every theory T and sentence φ:

$$T \vDash \varphi \quad \rightarrow \quad T \vdash \varphi$$

Proof *(contrapositive argument)*: Assume $T \nvdash \varphi$. Then $T \cup \{\neg\varphi\}$ is consistent. Extend it to a maximally consistent Henkin theory T^*. Build the term model $\mathfrak{M}$ of T^*. By the Truth Lemma, $\mathfrak{M} \vDash T^*$, hence $\mathfrak{M} \vDash T$ and $\mathfrak{M} \vDash \neg\varphi$. Therefore $T \nvDash \varphi$, proving the contrapositive. $\square$

Completeness (logic-level) is different from incompleteness (theory-level arithmetic).

Compactness Theorem

Theorem 5 (Compactness): A theory T is satisfiable iff every finite subset $T_0 \subseteq T$ is satisfiable.

Proof *(from completeness)*: The forward direction is immediate. For the converse, if T were unsatisfiable then $T \vDash \bot$. By completeness, $T \vdash \bot$. Any derivation is finite, so only finitely many premises are used; thus some finite $T_0 \subseteq T$ already proves $\bot$ — contradiction. $\square$

Example: Let $T = \mathrm{Th}(\mathbb{N}) \cup \{c > 0, c > 1, c > 2, \ldots\}$ with fresh constant c. Every finite subset is satisfiable in $\mathbb{N}$ by choosing c large enough. By compactness, T has a model, yielding a non-standard element larger than every standard numeral.

Downward Löwenheim–Skolem

Theorem 6: Let L be a language and let T be an L-theory with an infinite model. If L is countable, then T has a *countable* model.

More generally, if T has a model of size at least $\max(\aleph_0, |L|)$, then it has models of every infinite size κ with $\max(\aleph_0, |L|) \leq \kappa \leq |M|$.

Proof *(idea)*: Start with a set A of parameters of the desired size. Close A under witness-producing operations for existential formulas. The closure stays small, but is rich enough to satisfy the Tarski–Vaught test. Hence it forms an elementary submodel. $\square$

In the countable-language case, an infinite model always has a countable elementary submodel. So first-order logic cannot force an infinite model to be uncountable.

Upward Löwenheim–Skolem

Theorem 7: If an L-theory T has an infinite model, then for every infinite cardinal $\kappa \geq \max(\aleph_0, |L|)$, the theory T has a model of size at least κ.

Proof *(compactness argument)*: Add fresh constants c_i for $i < \kappa$ and add sentences $c_i \neq c_j$ for all distinct i, j. Every finite subset of this expanded theory is satisfiable in the original infinite model, because only finitely many distinct constants need interpretation. By compactness, the whole expanded theory has a model with at least κ distinct elements. $\square$

Note: With a standard cardinality argument, one can refine this to obtain a model of size exactly κ.

Skolem's Paradox and What LS Means

Skolem's paradox (1922): ZFC proves that uncountable sets exist, yet by Löwenheim–Skolem, if ZFC has any model at all, then it has a countable model.

Resolution: "uncountable" is relative to the model's membership relation. Inside that countable model, there is no *internal* bijection between the set and $\mathbb{N}$, even though from the outside the whole model is countable.

Consequences for first-order expressiveness:

- no first-order theory with an infinite model can force a unique infinite cardinality;
- pure first-order arithmetic cannot characterize $\mathbb{N}$ up to isomorphism;
- properties such as "the domain is finite" or "the domain is countable" are not first-order axiomatizable in the intended absolute sense.

What Is the Decision Problem?

Given a sentence φ, we can ask three different questions:

1. **Satisfiable?** Is there some structure $\mathfrak{M}$ with $\mathfrak{M} \vDash \varphi$?
2. **Valid?** Does every structure satisfy φ? Equivalently, $\vDash \varphi$.
3. **Provable?** Can we derive φ in a proof system? Equivalently, $\vdash \varphi$ or $T \vdash \varphi$.

Definition 13 (Decidable): A problem is *decidable* if there is an algorithm that halts on every input and always returns the correct yes/no answer.

FOL Validity Is Undecidable

Theorem 8 (Church–Turing, 1936): The *validity problem* for first-order logic is undecidable: there is no algorithm that decides, for every FOL sentence φ, whether $\models \varphi$.

Why PL is decidable:

- finitely many propositional variables
- finitely many truth assignments
- brute force always terminates

Why FOL is not:

- structures may be infinite
- domains may have arbitrary size
- quantifiers range over unbounded domains

Completeness does *not* imply decidability. It only says: if φ is valid, then some proof exists. It does not say how long proof search will take, or whether search terminates on invalid input.

Semi-Decidability: What Completeness Gives

Validity is semi-decidable. Enumerate all formal proofs. If φ is valid, then by completeness some proof of φ exists, so the search will eventually find it.

Unsatisfiability is semi-decidable. φ is unsatisfiable iff $\neg\varphi$ is valid. So a search for a proof of $\neg\varphi$ will eventually succeed when φ has no model.

Satisfiability is therefore co-semi-decidable. If φ is satisfiable, there need not be any finite certificate that a blind proof search will eventually find. Search may run forever even on satisfiable input.

For general FOL:

- validity is r.e. but undecidable;
- unsatisfiability is r.e.;
- satisfiability is co-r.e. and undecidable.

Gödel Incompleteness

Completeness and incompleteness concern different objects.

1. **Completeness theorem (logic-level):** for first-order logic, semantic consequence and derivability coincide: $T \vDash \varphi \to T \vdash \varphi$.
2. **Incompleteness theorems (theory-level):** for specific arithmetic theories, not every sentence true in the intended model $\mathbb{N}$ is provable inside the theory.

Completeness is about the proof system of first-order logic. Incompleteness is about the limitations of particular arithmetic axiomatizations.

First Incompleteness Theorem

Theorem 9 (Gödel I): Let $\mathcal{T}$ be a recursively axiomatized, consistent theory that can represent elementary arithmetic (for example, $\mathcal{T}$ extends Robinson arithmetic Q). Then there exists a sentence $G_{\mathcal{T}}$ such that:

- $\mathcal{T} \nvdash G_{\mathcal{T}}$, and
- if $\mathcal{T}$ is sound for arithmetic truth, then $\mathbb{N} \vDash G_{\mathcal{T}}$.

Note: There are arithmetic sentences true in the intended model $\mathbb{N}$ that are not provable in $\mathcal{T}$. Hence no single effective axiomatization captures all arithmetic truths.

Terminology checkpoint:

- "true" here means true in the standard naturals $\mathbb{N}$;
- "provable" means derivable inside the formal calculus of $\mathcal{T}$.

Second Incompleteness Theorem

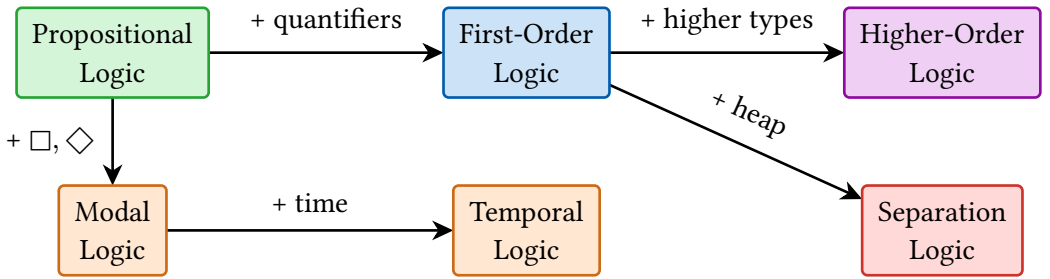
Theorem 10 (Gödel II): Under similar assumptions (in particular, sufficient arithmetic strength), if $\mathcal{T}$ is consistent then $\mathcal{T} \nvdash \mathrm{Con}(\mathcal{T})$.

Note: Here $\mathrm{Con}(\mathcal{T})$ is an arithmetized sentence expressing "there is no code of a contradiction-proof in $\mathcal{T}$." So a sufficiently strong consistent arithmetic theory cannot prove its own consistency by its own internal methods.

FM consequence: no fixed sufficiently expressive sound system can prove every true arithmetic/program property. Practical verification combines automated solvers with user-guided invariants, lemmas, and decomposition.

The Landscape of Logics

Classical PL and FOL are two points in a rich space of formal systems. Different verification tasks need different logics:



Modal Logic: $\Box\varphi$ ("necessarily") and $\Diamond\varphi$ ("possibly"). Kripke semantics: worlds + accessibility.

Temporal Logic: LTL ($\Box$, $\Diamond$, $\mathcal{U}$), CTL, CTL*. Model checking: $\Box\Diamond$ response.

Separation Logic: Spatial connectives ($*$, $*$) for heap memory. Powers Meta Infer, VeriFast.

Decidability Landscape

Logic / Fragment	SAT	VALID	Complexity
Propositional	Decidable	Decidable	NP-c / co-NP-c
Modal (K, S4, S5)	Decidable	Decidable	PSPACE-c
FOL (general)	Undecidable	Undecidable	r.e. / co-r.e.
Monadic FOL	Decidable	Decidable	NEXPTIME-c
Presburger $(\mathbb{N}, +)$	Decidable	Decidable	2-EXPTIME
Arithmetic $(\mathbb{N}, +, \times)$	Undecidable	Undecidable	Not decidable

Distinguish two failure modes: high complexity (decidable but expensive) versus undecidability (no terminating general procedure).

Why Theories?

Unrestricted FOL is too expressive for fully general algorithmic solving.

Definition 14: A *first-order theory* $\mathcal{T}$ fixes a signature and intended class of models.

- *$\mathcal{T}$-model*: structure satisfying all axioms of $\mathcal{T}$.
- *$\mathcal{T}$-satisfiability*: whether a formula has a model *inside* that class.

Verification tools therefore work in fragments/theories with known decision procedures, rather than arbitrary FOL over arbitrary structures.

SMT = SAT + Theories

Two components work together:

- **SAT layer:** handles Boolean structure ($\wedge$, $\vee$, $\neg$, case splitting, learning).
- **Theory layer:** handles domain constraints (arithmetic, arrays, bit-vectors, uninterpreted functions, ...).

Common SMT theories in verification:

- *LIA* (linear integer arithmetic): $0, 1, +, <, =$ over integers.
- *EUF* (equality with uninterpreted functions): congruence reasoning over symbolic functions.
- *Arrays*: read/write operators with array axioms.

SMT combines Boolean search with theory reasoning. Termination is guaranteed on the decidable fragments implemented by the solver.

From English to FOL

Translating natural language statements into FOL formulas is a core formalization task.
Here are examples over the signature of arithmetic: $\Sigma = \langle \{0, S, +, \times\}, \{<, =\} \rangle$.

1. *There is no largest natural number.*
   $\neg\exists x.\, \forall y.\, y \leq x$ (equivalently: $\forall x.\, \exists y.\, x < y$)
2. *Every prime greater than 2 is odd.*
   $\forall p.\, (\mathrm{Prime}(p) \wedge p > 2) \rightarrow \mathrm{Odd}(p)$
3. *For every natural number there is a greater one.*
   $\forall x.\, \exists y.\, x < y$
4. *Two distinct natural numbers cannot have the same successor.*
   $\forall x.\, \forall y.\, \neg(x = y) \rightarrow \neg(S(x) = S(y))$
5. *There are at least two natural numbers smaller than 3.*
   $\exists x.\, \exists y.\, \neg(x = y) \wedge x < S(S(S(0))) \wedge y < S(S(S(0)))$

Formalization checklist: (1) specify the domain, (2) fix quantifier order, (3) choose predicates/functions, (4) verify implication direction under quantifiers.

Exercises

Exercises: FOL Syntax and Semantics

1. Formalize the following in FOL:
   - "Every prime number greater than 2 is odd."
   - "There is no largest natural number."
   - "If f is injective and $A \subseteq B$, then $f(A) \subseteq f(B)$."
2. Determine free and bound variables, and whether each formula is a sentence:
   - $\forall x.\, (P(x) \to \exists y.\, Q(x, y))$
   - $\exists x.\, (x = y + 1)$
   - $\forall x.\, \forall y.\, (R(x, y) \to R(y, x))$
3. Convert to prenex normal form: $(\forall x.\, P(x)) \to (\exists y.\, Q(y))$
4. Explain precisely why $\exists x.\, \forall y.\, R(x, y) \vDash \forall y.\, \exists x.\, R(x, y)$ holds, but $\forall y.\, \exists x.\, R(x, y) \nvDash \exists x.\, \forall y.\, R(x, y)$. Give a concrete counterexample for the latter.

Exercises: FOL Proofs and Metatheorems

1. Prove the following using natural deduction (Fitch notation):
   - $\forall x.\,(P(x) \to Q(x)), \exists x.\,P(x) \vdash \exists x.\,Q(x)$
   - $\forall x.\,(P(x) \to Q(x)) \vdash (\exists x.\,P(x)) \to (\exists x.\,Q(x))$
   - $\forall x.\,P(x), \forall x.\,(P(x) \to Q(x)) \vdash \forall x.\,Q(x)$

2. Construct a countermodel to show that $\exists x.\,P(x) \land \exists x.\,Q(x) \nvDash \exists x.\,(P(x) \land Q(x))$.

3. $\star$ Using compactness, show that "the domain is finite" cannot be expressed by any *single* FOL sentence (or even by any *set* of FOL sentences). *Hint*: Consider the set of sentences φ_n asserting "there exist at least n distinct elements."

4. $\star$ The compactness theorem fails for *second-order* logic. Explain why, and give an example of a property SOL can express but FOL cannot.